Práctica 2 #dwjQuery

Esta práctica está dividida en "fases", que cada uno implemente la fase que le interese o pueda implementar.

FASE 1:

Hacer un plugin que convierta un formulario normal en un formulario Ajax.
Simplemente llamamos al plugin sobre el formulario y cuando se haga el "submit" que se envíe por ajax en vez de enviarse por el método común.

Requisitos:

1) Debes conseguir que en algún lugar aparezca el típico mensaje "cargando" mientras que se espera la respuesta a la solicitud ajax por parte del servidor.
2) Mientras que se está enviando el formulario debes desactivar el botón de submit, para que no se vuelva a enviar el formulario mientras que se recibe el resultado de la solicitud Ajax.

FASE 2:

Como segunda fase hacemos lo mismo que la fase 1 pero le agregamos validación.
Hacer un plugin que convierta un formulario normal en un formulario Ajax con validación.
El formulario se envía al servidor, que nos devuelve un JSON con un formato como este:

Si el formulario era válido)

```
{
        "valido": true
}
```

Si el formulario no era válido)

```
{
        "valido": false,
        "errores":"<ul>
                <li>No has indicado el nombre</li>
                <li>No has indicado el email</li>
                <\/ul>"
}
```

Nota: Las validaciones que hagas realmente me dan igual, la idea es que puedas procesar ese JSON en el cliente con jQuery.

Requisitos:

1) Al invocarlo, el plugin podrá recibir unas opciones como estas:

```
{
	url: "destilo.php",
	capaMensaje: "#msg",
	metodo: "get"
}
```

- En el campo "url" indicamos la url a la que hay que enviar el formulario.
- En el campo "capaMensaje" indicamos la capa donde se debe mostrar el mensaje de carga los mensajes cuando se reciba la respuesta del ajax, informando si el formulario era válido, de los posibles errores de validación, etc.

Las opciones por defecto que tendremos en el plugin son:

```
{
	url: false,
	capaMensaje: false,
	metodo: "post"
}
```

- Si url es false, entonces como URL tomamos el action del formulario.
- Si capaMensaje es false, entonces generamos un elemento en tiempo de ejecución del plugin ("al vuelo"), de cualquier tipo (me vale una DIV o un P, por ej). el elemento lo insertamos dentro del formulario y los mensajes los mostraremos dentro de ese elemento generado.
- metodo solo puede valer "post" o "get". Si no es cualquiera de estos valores, el método que se debe usar es "post".

FASE 3:

Como tercera fase se implementará la fase 2, más un mecanismo para que el formulario se envíe automáticamente cada cierto número de segundos.
Hacer un plugin que convierta un formulario normal en un formulario Ajax, con validación y además un mecanismo de "autosave".

Al invocar al plugin del formulario con "autosave" se genera un temporizador para que el formulario se envíe por Ajax pasados unos segundos. El tiempo será configurable por el objeto de opciones del plugin, de modo que el objeto de opciones recibirá los mismos datos que la Fase2, más un dato llamado "segundos". Por defecto segundos tomará el valor de 20.

El mecanismo es el siguiente.

1) Invocas al plugin y generas el temporizador para que se auto-envíe pasados x segundos.
2) Si pasados esos x segundos no lo hemos enviado nosotros manualmente (pulsando el botón de submit), el formulario se autoenvía solo y se establece otra vez el temporizador para

enviarlo de nuevo automáticamente pasados los segundos definidos en las opciones del plugin. 3) Si antes de que se llegue a producir el auto-envío, osea, antes de completarse los segundos indicados al invocar al plugin, el usuario envía manualmente el formulario pulsando el botón de submit, el formulario se tiene que enviar por Ajax, pero el temporizador de autoenvío se debe parar. Una vez recibida la respuesta de Ajax, se vuelve a establecer el temporizador de autoenvío.

Nota: el temporizador lo puedes implementar con las funciones setTimeout() o setInterval(), nativas de Javascript.

Fase 4:

Generar unos eventos personalizados en el plugin, cuando empiece y finalice el envío del formulario por ajax. Esos eventos se generan con trigger() desde el código del plugin. Desde fuera del código del plugin, para alguno de los formularios ajax que tengas en la página, definirás dos manejadores de eventos para esos eventos personalizados, para hacer cosas cuando empieza el envío del formulario y cuando finaliza.

Como sugerencia puedes crear una capa "overlay" que aparezca en el medio de la pantalla y con el típico símbolo del loading (esa ruedita que se va moviendo para expresar que la página está cargando). Cuando se comience a enviar el formulario se muestra y cuando finalice, se oculta la capa loading.

Nota: Los eventos personalizados los expliqué en la clase extra de eventos. No sé si los visteis de nuevo en las clases de plugins.

Suerte! y disfrutar codeando!